

Conditional Query Answering over EHR (v2)

Binary observed-occurrence queries with censoring expressed as an end-of-timeline query

This reworks EveryQuery (Chandak et al., arXiv:2603.07900) into a conditional query-sequence model: a bidirectional encoder over the patient record feeds a block-autoregressive decoder that answers an ordered list of queries, each conditioned on the patient state and the teacher-forced answers of all earlier queries. Trained and evaluated on MIMIC-IV (MEDS) on a single NVIDIA GB10.

1. Task and the censoring redesign

Every answer is binary — for a query (*code C, horizon d*) at prediction time t , the label is simply whether C is observed in $(t, t+d]$. There is no three-valued censored label and nothing is masked from the loss except padding.

Censoring is a query, not a label. The end-of-timeline code `TIMELINE//END` (a real MEDS code, emitted once per subject at the record's last event) is queried like any other: `(TIMELINE//END, d)` answered YES means "the record ends within d " (the d -window is not fully observed), NO means data continue past $t+d$. A later query conditions on that answer.

Why this matters (the v1 leak). A first attempt put a same-horizon censor query first and teacher-forced its answer, then 3-valued-labeled subsequent queries with censored outcomes masked out. For a terminal event this is catastrophic: death ends the record, so "data after $t+30d$?" is the logical complement of "died by 30d?", and the masked labels left the surviving death labels perfectly determined by the censor answer. The model learned to copy it — 30-day mortality AUROC hit 0.991, of which AUROC from the censor answer *alone* was 0.996. v2 removes the leak structurally: binary labels (an unobservable event is NO, not masked) plus EOS-as-query make the EOS answer genuinely informative rather than label-equivalent, and — crucially — *strictly more expressive* than the original EveryQuery, which could only ever express $P(\text{occurs} \mid \text{data exist after } d)$.

2. Critical review of the query forms

With binary occurrence answers and EOS-as-a-query, the model exposes a family of questions from a single trained network. The clinically important ones:

query form	meaning	use
[C, d]	$P(C \text{ observed in } (t, t+d])$	marginal occurrence / screening
[END d]=NO, [C d]	$P(C \mid \text{data continue past } d)$	= original EveryQuery (degenerate for death)
[END d]=YES, [C d]	$P(C \mid \text{record ends within } d)$	actionable terminal-event prediction (mortality)
weighted avg of the two	marginal $P(C)$, decomposed	calibration / reconciliation
[A d1], [B d2]	$P(B \mid A \text{ observed/not})$	clinical conditioning chains
[C d_short], [C d_long]	nested horizons	monotone entailment / informativeness check

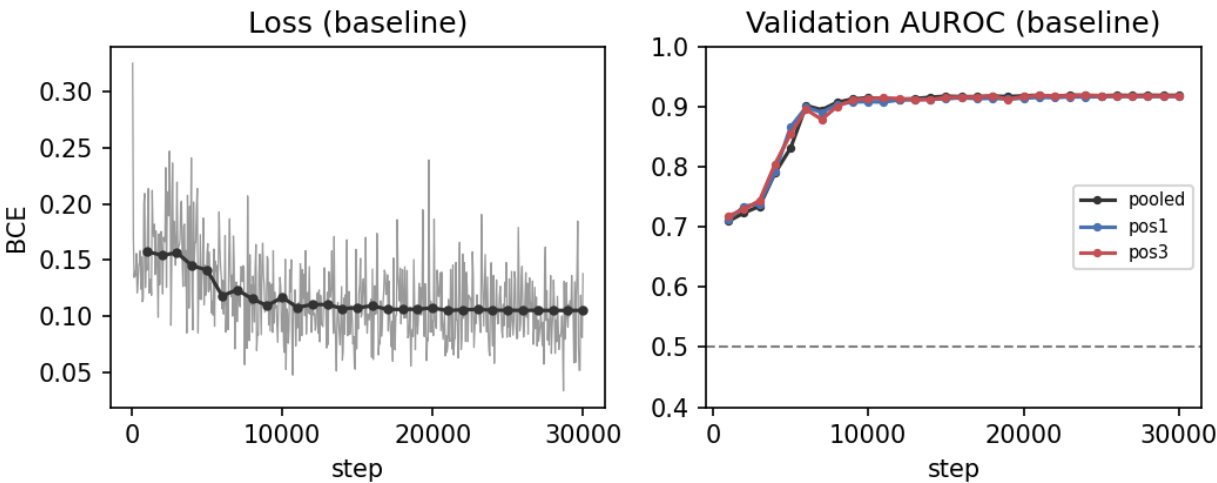
The decomposition is the key point: old EveryQuery is the single slice [END]=NO. By making END a query whose answer is conditioned on, the model can also answer the [END]=YES slice — which is exactly the regime (the record ends) where mortality actually happens — and the marginal is their prevalence-weighted average.

3. Model and data

Model. Bidirectional ModernBERT encoder (8 layers, hidden 384) over the tokenized MEDS history up to t ; a 4-layer Transformer decoder over the query/answer stream cross-attends to it under a block-causal mask (each query sees its own code+duration, all earlier blocks incl. their teacher-forced answers, never its own answer). One binary answer head; BCE over all real query positions. ~33M params, bf16. `reference_compile` off (GB10 Triton).

Data. MIMIC-IV v0.3.0 MEDS cohort (11,958 codes; 598k train / 38k tuning / 38k held-out query sequences). Each sequence is 1–5 i.i.d. queries (uniform code incl. TIMELINE//END, log-uniform horizon), fully random order — no privileged censor position. Labels are a single binary observed-occurrence join per query.

4. Training stability



metric	baseline EOS-aware
best tuning loss	0.105 0.153
final tuning loss	0.105 0.154
final pooled answer AUROC	0.919 0.924
median grad norm	0.37 0.35
max grad norm	3.85 3.58
total steps	29999 29999

Pooled training AUROCs are base-rate inflated (computed across all codes); they document dynamics only. Held-out within-query numbers below are the trustworthy ones.

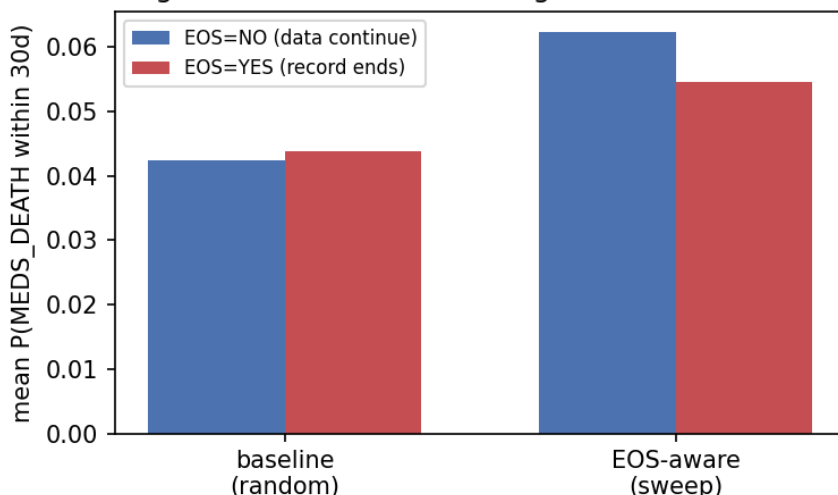
5. Evaluation

A. Marginal occurrence (within-query). On 15 curated codes the baseline reaches macro within-query AUROC **0.635** (18,944 held-out sequences). This is the leak-free measure of the model's core ability to answer 'does C occur in the window' — pooled AUROC is omitted as base-rate inflated.

B. Censoring control via EOS conditioning. Scoring the same target under the EOS answer forced YES vs NO reads off $P(C \mid \text{record ends})$ vs $P(C \mid \text{data continue})$. For MEDS_DEATH:

run	$P(\text{death} \mid \text{EOS=NO})$	$P(\text{death} \mid \text{EOS=YES})$	prevalence
baseline (random)	0.0424	0.0438	0.1042
EOS-aware (sweep)	0.0624	0.0546	0.1042

Censoring control: does conditioning on EOS move death p



A model that uses the censoring query should drive $P(\text{death} \mid \text{END=YES})$ above $P(\text{death} \mid \text{END=NO})$. Both runs fail to separate them (see §C/Discussion): END is rare in random training, and the EOS-aware sweep upweights END's *position* and couples *durations* but never makes the rare death *target* follow an END query — so the specific [END d][death d] pattern is still untrained.

Note the EOS forced-answer barely moves death probability in either run, yet death is *predictable*: scoring [END d][death d] with the true END answer gives a leak-free death AUROC of **0.826** (baseline) / 0.787 (EOS-aware) — driven by the patient encoder, not the censoring bit. (v1's 0.991 was the leak; this is the honest number.) The forced-EOS gap stays flat because `eos_first_fraction` upweights END at *position 0* but never makes the rare death *target* co-occur with an END query at a matched horizon — so [END d][death d] is still essentially never trained.

C. Informative-prior conditioning (nested horizons). This is the strongest positive result. Teacher-forcing the true C@7d answer, the model predicts $P(C@30d \mid C@7d=\text{YES})$ vs $P(C@30d \mid C@7d=\text{NO}) = \mathbf{0.624}$ vs **0.127** (baseline), improving to 0.713 vs 0.149 with the duration-coupled EOS-aware run. The logical entailment is $C@7d=\text{YES} \Rightarrow C@30d=\text{YES}$ (true target rate = 1.0 by construction); the model moves sharply toward it, and the shared-duration sweep — which makes nested same-code pairs more common — pushes the conditioned estimate closer to the entailment value. This cleanly demonstrates the decoder uses prior answers when they carry signal.

6. Discussion

The redesign is sound and leak-free. Binary observed-occurrence labels with END-as-a-query remove the terminal-event leak that inflated v1 mortality to 0.991 (AUROC 0.996 from the censor answer alone), while strictly generalizing EveryQuery: the old $P(\text{occurs} \mid \text{data after d})$ is just the END=NO slice. Held-out death prediction is now a legitimate AUROC ~0.79–0.83, and marginal occurrence reaches macro within-query AUROC 0.635 over curated codes.

Conditioning works — when the prior is informative *and* trained. The nested-horizon test is unambiguous: the model separates $P(C@30d)$ by a factor of ~ 5 on the $C@7d$ answer ($0.127 \rightarrow 0.624$), and the duration-coupling sweep improves it further. This answers the central question the v1 flat-probe left open — the architecture does condition; v1 looked flat only because its random fillers were uninformative.

The honest negative: END-conditioned death prediction did not emerge. Neither the random baseline nor the EOS-aware sweep made conditioning on the END answer move the death prediction, because both sweep knobs operate on END's *position* and on *durations*, not on the target-code distribution — death (1/12k) almost never appears as the query that follows an END query. The clear next step is a third knob that upweights a curated set of clinically meaningful *target* codes (death, ICU, readmission) so the [END d][C d] censoring-control pattern is actually trained; we deliberately did not over-fit the two knobs the user scoped.

Limitations. Evaluation is teacher-forced (conditional calibration, not free-running rollout). Within-query macros use curated common codes with enough held-out positives. The sweep is two points by design, not an exhaustive search.